MediVault

Medicine Inventory and Expiry Management System

Project explanation report for academic presentation

Project type	Desktop and browser-based inventory application
Language	Python 3
Database	SQLite3
Interfaces	Tkinter, Streamlit, and custom Flask web UI
Testing	Python unittest suite
Storage	Local SQLite database at data/medivault.db

Prepared as a clear guide for explaining the project to a teacher, examiner, or evaluator.

1. Project Overview

MediVault is a medicine inventory management system designed for small pharmacies, clinics, medical stores, and personal medicine cabinets. It solves the problem of managing medicines only by name, which is unsafe because the same medicine can arrive in multiple batches with different expiry dates and quantities.

The system stores each batch separately, monitors expiry risk, calculates low-stock conditions, supports FEFO dispensing, records stock movements, generates reports, and creates database backups.

The core design separates the user interface, business logic, and database access. This makes the application easier to test, maintain, and extend.

One-sentence explanation:

MediVault is a batch-aware medicine inventory system that helps a store know what stock it has, which stock should be used first, which stock is expiring, and why the quantity changed.

2. Problem Statement and Objectives

A basic medicine list is not enough for real inventory control. A store needs to know the exact batch, expiry date, supplier, purchase price, and remaining quantity. Manual records can cause expired medicines to remain in stock, incorrect quantities, missed replenishment, and no explanation for stock changes.

Project objectives

• Maintain a searchable medicine catalog.

• Track multiple independent batches for one medicine.

• Detect expired and near-expiry batches.

• Warn when total medicine stock is below its threshold.

• Use FEFO to dispense the batch that expires first.

• Record every stock movement with before and after quantities.

• Provide reports, CSV export, backups, and multiple user interfaces.

Expected benefits

The system reduces expiry-related waste, improves stock visibility, makes stock changes traceable, and demonstrates practical Python concepts such as classes, validation, SQL, file handling, exception handling, and automated testing.

3. Technology Stack

Technology	Why it is used
Python 3	Main programming language; readable, fast to develop, and suitable for desktop, web, database, and testing work.
SQLite3	Embedded relational database; no separate database server is required for a local application.
Tkinter and ttk	Original desktop interface with navigation, dialogs, tables, themes, and keyboard shortcuts.

Technology	Why it is used
Streamlit	Python-based browser interface with dashboard, forms, alerts, stock operations, reports, and downloads.
Flask	Custom HTML/CSS/JavaScript browser interface with a REST-style API.
HTML, CSS, JavaScript	Custom web UI layout, responsive styling, search, navigation, notifications, and CSV download.
unittest	Automated tests for validation, CRUD, expiry boundaries, stock rules, FEFO, reports, and backups.
ReportLab	Used to generate this project explanation PDF.

Why SQLite instead of a larger database?

MediVault is designed as a local offline-first application. SQLite is included with Python, stores everything in one file, supports relationships and constraints, and is simple to back up. A future multi-user online version could move the data layer to PostgreSQL.

4. How the Application Runs

The application has three available interfaces. All interfaces use the same service layer and the same SQLite database, so data created through one interface can be read by another.

Interface	Command	Purpose
Desktop	python main.py	Tkinter application with the full original desktop workflow.
Streamlit	py -3.14 -m streamlit run app.py	Python-generated browser interface.
Custom web	py -3.14 web_app.py	Custom HTML/CSS/JavaScript browser interface.

Shared request flow

```
User interface -> service layer -> DatabaseManager -> SQLite database
```

Example: when a user adds a medicine, the form collects the fields, the service validates the name and threshold, DatabaseManager executes a parameterized INSERT statement, and SQLite stores the record in the medicines table.

5. Project Architecture

Layer	Main files	Responsibility
Presentation	ui/, app.py, web_app.py, templates/, static/	Displays screens, forms, dashboards, tables, alerts, and reports.
Business services	services/	Contains medicine CRUD, stock movement, FEFO, expiry classification, reports, and aggregation rules.
Data access	database/connection.py	Opens SQLite, enables foreign keys, runs queries, commits statements, and supports transactions.
Schema	database/schema.py	Creates tables, relationships, indexes, constraints, and default settings.

Layer	Main files	Responsibility
Models	models/	Dataclasses representing medicines, batches, and stock transactions.
Utilities	utils/	Validation, backup, CSV import/export, and logging.
Tests	tests/	Automated checks for important business rules.

Why use a service layer?

The UI should not contain database rules everywhere. Services provide one controlled place for validation and business logic. For example, desktop, Streamlit, and Flask can all call InventoryService.add_stock(), so stock rules remain consistent across interfaces.

6. Database Design

The main relational tables are medicines, batches, stock_transactions, suppliers, and application_settings.

Table	Important fields	Purpose
medicines	medicine_id, name, generic_name, category, unit_type, min_stock_threshold	Stores the medicine catalog.
batches	batch_id, medicine_id, batch_number, expiry_date, current_quantity, prices	Stores stock as independent production or purchase lots.
stock_transactions	transaction_type, quantity_changed, previous_quantity, new_quantity, reason	Stores the stock audit history.
suppliers	supplier_id, name, contact, address	Stores supplier information.
application_settings	key, value	Stores preferences such as theme and currency.

Database safeguards

• Foreign keys are enabled with PRAGMA foreign_keys = ON.

• A medicine cannot have duplicate batch numbers because of UNIQUE(medicine_id, batch_number).

• New schemas enforce nonnegative quantities and prices with CHECK constraints.

• SQL parameters use ? placeholders instead of string concatenation.

• Stock update plus audit insert runs in a transaction and rolls back together on failure.

Important design note: the current schema uses cascade deletion for medicines and batches. That avoids orphan rows, but a production compliance system should use archival or soft deletion if historical audit records must never be removed.

7. Medicine and Batch Workflow

Adding a medicine

The user enters the name, generic name, category, dosage form, manufacturer, description, and low-stock threshold. MedicineService checks that the name is not empty and the threshold is not negative. The record is saved in data/medivault.db, inside the medicines table.

Registering a batch

The user selects a medicine and enters a batch number, manufacturing date, expiry date, purchase date, quantity received, prices, supplier, and notes. InventoryService checks the dates, quantity, prices, and batch number. The new batch starts with current_quantity equal to quantity_received, and an INITIAL transaction is created.

Why batch-level stock matters

If Paracetamol has one batch expiring in five days and another expiring in ninety days, the application must not treat them as one anonymous quantity. Batch records allow safe expiry decisions and FEFO dispensing.

8. Expiry and Low-Stock Logic

Expiry status is calculated from the ISO date format YYYY-MM-DD:

```
days_remaining = expiry_date - today
```

Status	Condition	Meaning
EXPIRED	days < 0	The batch is past its expiry date.
CRITICAL	0 to 7 days	Immediate attention is needed.
WARNING	8 to 15 days	Plan usage or replenishment.
UPCOMING	16 to 30 days	Monitor upcoming expiry.
NORMAL	More than 30 days	Healthy remaining shelf life.

Low-stock calculation

For each medicine, the system sums current_quantity across all of its batches. If the total is lower than min_stock_threshold, the medicine appears as low stock on the dashboard and alert pages.

The custom web interface also supports an Enable alerts control. After the user enables it, a changed low-stock alert can produce an in-page popup, a short browser sound, and a browser notification when permission is granted.

9. FEFO Dispensing

FEFO means First Expiry, First Out. It is more appropriate for medicines than simply removing from the oldest database row or the newest stock entry.

Algorithm

```
1. Find non-expired batches for the selected medicine with current_quantity > 0. 2.
Sort by expiry_date ascending. 3. Check that the total available quantity is
sufficient. 4. Remove as much as possible from the earliest batch. 5. Continue with
later batches until the requested quantity is complete. 6. Record an audit transaction
for every affected batch.
```

The availability check and deductions occur inside one database transaction. If the requested amount is not available, the operation raises an error and no partial deduction is committed.

Example: if batch A has 80 units and expires in 5 days, while batch B has 200 units and expires in 90 days, dispensing 100 units removes 80 from A and 20 from B.

10. Stock Audit and Data Integrity

Every stock change creates a stock_transactions row. The record contains the type of operation, quantity changed, previous quantity, new quantity, reason, batch, medicine, and timestamp.

The system supports four transaction types: INITIAL, ADDED, REMOVED, and ADJUSTED.

Stock operations are atomic. The quantity update and transaction insert are inside one transaction context. If the audit insert fails, the quantity update is rolled back, preventing the database from showing a stock change with no explanation.

The database manager also uses a re-entrant lock and a thread-compatible SQLite connection so the shared data layer can serve Flask requests safely.

11. Reports, CSV, and Backups

Reports

• Complete inventory report with medicine, batch, expiry, quantity, price, and supplier.

• Expiry report for expired and near-expiry batches.

• Low-stock report grouped by medicine threshold.

• Stock movement report filtered by date.

• Valuation report using current quantity multiplied by purchase price.

CSV

The project exports medicines, batches, and transactions as CSV. UTF-8 with a BOM is used for compatibility with Excel. Medicines can also be imported from CSV with validation.

Backup

A backup is a timestamped copy of the SQLite database. Metadata is written as JSON. Before restore, the current database receives a safety copy. This makes it possible to return to a previous snapshot without immediately losing the current state.

12. Validation, Errors, and Security

• Medicine names cannot be empty.

• Batch numbers cannot be empty.

• Expiry must be after manufacturing date.

• Quantities must be whole, nonnegative values.

• Prices must be finite and nonnegative.

• Stock removal cannot exceed available stock.

• Parameterized SQL reduces SQL injection risk.

• Service exceptions are shown as user-facing error messages in forms.

This is a local application and is not an authentication system. A production online deployment would need login, role-based permissions, HTTPS, a managed database, and a proper deployment server.

13. Testing

The project uses Python unittest. The tests use separate temporary SQLite databases so normal test execution does not overwrite the user database.

Test area	What is checked
Validators	Names, quantities, prices, dates, date ranges, batch numbers.
Medicines	Create, update, search, category filtering, threshold validation.
Batches	Creation, duplicate batch prevention, date checks, update protection.
Expiry	Boundary values such as -1, 0, 7, 8, 15, 16, 30, and 31 days.
Stock	Add, remove, correction, negative-stock prevention, FEFO, audit rows.
Reports	Inventory, low-stock, valuation, and CSV export.
Backup	Database snapshot creation, listing, restore path, and CSV import.

The current project validation result is 38 tests passed, with all Python modules compiling successfully.

14. Suggested Demonstration Sequence

Use this sequence during the presentation:

Step	Demonstration	What to explain
1	Open the dashboard	The dashboard aggregates live database values.
2	Open Medicines	Search, categories, thresholds, and catalog records.
3	Add or show a medicine	The record is saved in the medicines table.
4	Open Batches	One medicine can have multiple lots and expiry dates.
5	Open Alerts	The application classifies expiry risk and low stock.
6	Use FEFO dispense	The earliest-expiring valid batch is consumed first.
7	Open Stock audit	Before, after, quantity, reason, and timestamp are visible.
8	Open Reports	Inventory can be exported to CSV.
9	Open Settings or backup workflow	The SQLite database can be snapshotted and restored.
10	Show tests	Automated tests verify the important business rules.

15. Viva Questions and Short Answers

Question	Answer
Why did you use batch-level inventory?	Because batches have different expiry dates, suppliers, prices, and quantities. Batch tracking makes FEFO and expiry control possible.
Why SQLite?	It is included with Python, works offline, stores the project in one file, supports relational constraints, and is easy to back up.

Question	Answer
What is FEFO?	First Expiry, First Out. The system dispenses from the valid batch with the earliest expiry date first.
How do you prevent negative stock?	The service checks available quantity, the database has nonnegative checks for new schemas, and stock operations are transactional.
How is expiry status calculated?	The system subtracts today's date from the stored ISO expiry date and maps the number of days to five urgency tiers.
How do you maintain an audit trail?	Every initial entry, addition, removal, and correction inserts a stock transaction with previous and new quantities.
What happens if an operation fails?	The transaction rolls back, so the quantity update and its audit row do not become inconsistent.
Can the system be used online?	The Flask and Streamlit interfaces can run in a browser. For production, authentication and a managed database would be required.
What did you learn?	Layered architecture, Python classes, SQL relationships, validation, transactions, algorithms, testing, file handling, and UI design.

16. Limitations and Future Improvements

• Add authentication and user roles for online use.

• Move from local SQLite to PostgreSQL for multi-user deployment.

• Use soft deletion or archival for legally immutable audit history.

• Add scheduled background notifications and email or WhatsApp alerts.

• Add barcode or QR-code scanning.

• Add supplier purchase orders and sales invoices.

• Add charts for monthly stock movement and expiry loss.

• Add PDF report export from the application itself.

These are future improvements; the current implementation already demonstrates the main inventory, expiry, FEFO, audit, reporting, backup, and testing workflows.

17. Final Presentation Summary

MediVault is not just a form that stores medicine names. It is a structured inventory system. The database separates medicines from batches, the services enforce rules, the FEFO algorithm reduces expiry waste, the alert system highlights risk, and the audit table explains every stock movement. Multiple interfaces demonstrate that the same business logic can support desktop and browser applications.

The strongest technical points to emphasize are: batch-level modeling, FEFO allocation, transactional stock operations, SQLite relational design, service-layer separation, input validation, CSV and backup utilities, notification support, and automated tests.